University of the Armed Forces

DEPARTMENT OF COMPUTER SCIENCE

Object-Oriented Programming

IEEE 830 Requirements

Edison Lascano PhD

Guañuna Ajila Cristopher John

Jaramillo Naupari Julian Alfonso

Lanchimba Carvajal Cristopher Daniel

1. Introduction

This Software Requirements Specification (SRS) document describes the main features, goals, and functions of a computer system for the daily control of income and expenses of a company that offers different products and services. The system aims to make it easy to record money movements, keep financial information organized, and create daily reports that show the status of the cash register.

Today, keeping an organized control of income and expenses is important to know how money moves inside a company. For this reason, we propose to build a system that allows recording income obtained from selling products and providing services, as well as expenses generated by different costs. In this way, we expect to reduce mistakes in the records, improve the organization of information, and make it easier to calculate the daily balance.

1.1. Purpose

The purpose of this document is to set and describe the requirements of the daily income and expense control system, defining its main functions, its scope, and the benefits expected from its implementation.

This document is for the project development team, the teacher in charge of the Object-Oriented

Programming (OOP) course, and the people related to the operation of the company that will use the system. Also, it will serve as a guide during the analysis, design, programming, and testing stages to check that the software meets the identified needs.

1.2. System Scope

The system, called Daily Cash Control System (SCDC), has the goal of recording and controlling the daily income and expenses of a company that offers different products and services.

The system will allow users to record cash movements, calculate total income and expenses, get the daily balance, and generate reports to check financial information.

Functions that the system will do:

- Record income from sales and services.
- Record the expenses of the company.
- Calculate income, expenses, and daily balance.
- Generate cash reports.

Functions that the system will not do:

- Keep the complete accounting of the company.
- Manage inventories or employee payrolls.
- Make bank transfers or tax declarations.

The goal of the system is to improve the organization of financial movements, reduce mistakes in calculations, and make the daily control of the company's money easier.

1.3. Definitions, Acronyms, and Abbreviations

Term	Definition
SRS (ERS)	Software Requirements Specification. Document that describes the functions, features, and limits of a software system.

Term	Definition
OOP (POO)	Object-Oriented Programming. Programming paradigm that organizes software using classes and objects.
SCDC	Daily Cash Control System (*Sistema de Control Diario de Caja*). Proposed name for the project system.
Software System	Set of computer programs and components that allow doing specific tasks.
Income	Amount of money that the company receives from selling products or providing services, among other activities.
Expense	Amount of money that leaves the company to cover costs or other obligations.
Cash Movement	Record of money coming in or going out during a specific period.
Initial Balance	Amount of money available in the cash register at the start of the day.
Final Balance	Amount of money left in the cash register at the end of the day, based on the initial balance and recorded movements.
Daily Report	Summary of the income, expenses, and balance for one day of activity.

Term	Definition
Software Requirement	Condition, function, or feature that the system must meet.
Class	Structure that defines the attributes and behaviors of a type of object in OOP.
Object	Instance of a class that represents a specific element of the system.

1.4. References

This subsection will show a complete list of all documents referenced in the SRS.

1.5. Document Overview

This document is organized into sections that step-by-step describe the requirements of the Daily Cash Control System (SCDC).

The first section presents the introduction, the purpose of the document, the system scope, the definitions used, the references, and the general organization of the SRS.

The next sections will describe in more detail the general features of the system, the types of users, the functions it must fulfill, the functional and non-functional requirements, the constraints, and the conditions that the software must meet.

Finally, the specification will serve as the base for the design of the classes and their relationships, the implementation of the program, and the execution of tests that check that the system meets the company's daily income and expense control needs.

2. Overall Description

FlowDesk is a work order management and cash control system developed for Imprenta Tumbaco as part of the Object-Oriented Programming course at the University of the Armed Forces ESPE. Its purpose is to organize information about customers, products, services, work

orders, payments, income, and expenses in a single application.

The system aims to make it easy to track work orders, check pending balances, and search for information. It offers a simple interface with basic management tools, a dashboard, and report generation, without including advanced accounting features.

2.1. Product Perspective

FlowDesk is an independent application designed to support the daily activities of a print shop. It centralizes business information and allows users to manage work orders and financial transactions through a simple interface.

The system includes a side menu and a dashboard to view work orders according to their status. According to the project documentation, it does not depend on any specific external application.

General structure:
Print shop users → FlowDesk → Centralized information

2.2. Product Functions

FlowDesk will offer the following main functions:

- **Customer management:** Record and search customer information.
- **Product and service management:** Record products and services by category.
- **Work order management:** Create orders with a unique ID and update their statuses: Pending, Approved, In Process, Finished, and Delivered.
- **Income and expense management:** Record daily financial transactions.
- **Payment management:** Record full or partial payments—including deposits—and check pending balances. Payments can be made in cash or by bank transfer.
- **Dashboard:** View work orders grouped by status.
- **Reports:** Generate daily, weekly, and monthly reports on work orders and financial transactions.
- **Login and logout:** Access to the system for authorized users.

2.3. User Characteristics

FlowDesk will have three main types of users:

User Type	Main Activities
Employee	Manage information about customers, products, services, and work orders.
Cashier	Record income, expenses, payments, and pending balances.
Manager	Review reports on work orders and financial transactions.

Users are expected to have basic computer skills. The system will have a simple interface that does not require advanced technical or accounting knowledge.

2.4. Constraints

- The system will focus on the main activities of the print shop and will not include advanced accounting functions.
- The project will use Object-Oriented Programming (OOP).
- Each work order must have a unique ID and one of the five defined statuses.
- The payment system must support cash, bank transfers, partial payments, and pending balances.
- The application must include basic login and logout functions.
- Reports will be limited to daily, weekly, and monthly information.
- Requirements regarding programming language, operating system, database, and hardware have not yet been specified in the project documents.

2.5. Assumptions and Dependencies

- FlowDesk assumes that the print shop will provide accurate information about customers, products, services, work orders, and financial transactions.
- Employees, the cashier, and the manager are expected to use the system to keep records up to date. The accuracy of payment balances and reports will depend on the information

entered by users.

- The application will require a compatible computer. The specific operating system, hardware requirements, and need for an internet connection must be defined during development.
- Changes in print shop procedures or business needs may require system updates.

2.6. Future Requirements

Future versions of FlowDesk could include extra features to improve business management, such as:

- **Inventory management:** tracking materials and products.
- **Supplier management:** recording suppliers and purchases.
- **Advanced reports:** including more detailed financial information and statistics.
- **Change history:** recording modifications in work orders and financial transactions.
- **Expanded access control:** assigning permissions according to the responsibilities of each user.
- **Notifications:** alerts to users about pending orders and jobs ready for delivery.
- **Backups:** protecting information against possible data loss.
- **External integrations:** connecting the system with billing tools or other applications, as needed.

These improvements will depend on the future needs of the print shop and the available resources.

3. Specific Requirements

This section sets the requirements of FlowDesk for managing customers, products, services, work orders, payments, and cash movements of Imprenta Tumbaco. Its content forms the base for designing, building, and verifying the system.

The requirements describe observable behaviors and are identified with unique codes: **IE** for external interfaces, **RF** for functional requirements, **RNF** for non-functional requirements, and **RD** for design constraints. Functional priorities match the project backlog. Conditions expressly identified as proposals require validation before being added to the requirements baseline.

The specification must keep the following characteristics:

- **Correctness:** Each requirement must answer a project need and respect its scope.
- **Lack of ambiguity:** Terms must have a defined meaning. Production status, pending balance, and movement balance represent different concepts.
- **Completeness:** Requirements must set the inputs, validations, and expected results, including how invalid data is handled.
- **Consistency:** Requirements must not contradict each other or produce differences between related orders, payments, and movements.
- **Classification:** Each functional requirement must keep its priority and source reference.
- **Verifiability:** Compliance must be checked through inspection, demonstration, or tests with clear results.
- **Modifiability:** Changes must keep the IDs and update the affected requirements and references.
- **Traceability:** Each requirement must relate to its source and to the design and test elements that allow verifying it.

3.1. External Interfaces

IE-01. User interface

The system must provide forms and queries in Spanish to access customers, products, services, orders, payments, cash, and reports. Each view must identify the operation and the selected record.

When an input fails a validation, the system must point out the affected field, explain the reason, and keep the other entered data. Searches with no matches must show a message indicating no results were found.

IE-02. Hardware interface

The system must allow its operation using a screen, keyboard, and pointing device (mouse). The minimum computer specifications must be set when confirming the execution platform.

IE-03. Software interfaces

The application must exchange the necessary information with the storage mechanism to load and save its records. If a read or write operation fails, it must report the problem without showing

the operation as completed.

The format, location, and storage technology require validation.

IE-04. Communications interfaces

For the proposed local mode, administrative operations must run without depending on an Internet connection. The initial scope does not include direct integration with banking services or electronic billing platforms.

3.2. Functions

Functional requirements are organized by user goals to make it easier to match them with the backlog user stories and use cases.

RF-01. Login and logout

The system must validate the username and password before enabling administrative functions. If the credentials are wrong, it must reject access with a message that does not reveal which piece of data caused the rejection.

When logging out, it must return to the login screen and lock protected functions until a new login.

- **Source:** US008.
- **Priority:** Medium.
- **Acceptance criteria:** Valid credentials allow access; invalid ones do not enable administrative functions. After logging out, these functions remain locked.

RF-02. Manage customers

The system must allow registering, searching, and viewing customers using their name, ID number, and contact data. Each record must keep an internal ID that allows linking it with its orders.

The search must accept ID number and name. Whether contact data is mandatory and the uniqueness rules for the ID number must be validated with the business.

- **Source:** US001.
- **Priority:** High.

- **Acceptance criteria:** A registered customer can be found and selected when creating an order. A search with no matches shows an empty list and a message of no results.

RF-03. Manage products and services

The system must allow registering and viewing products and services with name, description, and category, plus prices and quantities when they apply to the registered item.

Changes to reference prices in the catalog must not change the agreed prices saved in existing orders.

- **Source:** US004 and project definition.
- **Priority:** High.
- **Acceptance criteria:** Registered items can be found by their category. Changing a price in the catalog keeps the amounts of previously created orders unchanged.

RF-04. Create work orders

The system must assign a unique code to each order and link it with a registered customer. It must save the job description, the requested products or services, the quantities, and the agreed unit prices.

The subtotal of each detail must be calculated as quantity multiplied by unit price. The total must correspond to the sum of the subtotals.

The number of allowed details and whether fractional quantities or prices equal to zero are accepted require validation.

- **Source:** US002.
- **Priority:** High.
- **Acceptance criteria:** An order with two units at 15.00 and one unit at 20.00 shows a total of 50.00 and a code different from any other order.

RF-05. View orders

The system must allow searching for an order using its code and showing the linked customer, job details, status, total, recorded payments, and pending balance.

- **Source:** US002 and project definition.
- **Priority:** High.

- **Acceptance criteria:** Searching for an existing code shows the information of the corresponding order. A non-existent code does not return another order or change records.

RF-06. Update order statuses

The system must allow representing the statuses: *Waiting (Pending)*, *Approved*, *In Process*, *Finished*, and *Delivered*.

Updating the status must keep the commercial and financial data of the order. Changing to *Delivered* must not automatically delete or modify its debt.

The sequence of transitions, going back to previous statuses, and change conditions require validation with the business.

- **Source:** US006.
- **Priority:** High.
- **Acceptance criteria:** When updating an order, the query shows the new status and keeps its total, its payments, and its balance.

RF-07. Show orders dashboard

The system must show the number of orders corresponding to each status and allow viewing the orders using filters by status.

The counts and lists must be obtained from the same set of records and apply the same selection criteria.

- **Source:** US003.
- **Priority:** High.
- **Acceptance criteria:** If there are three orders *In Process*, the dashboard shows three and the corresponding filter returns those exact same orders.

RF-08. Record payments and deposits

The system must allow recording full or partial payments linked to an order. Each payment must keep its ID, date, amount, and method.

The supported methods are *Cash* and *Bank Transfer*. Allowing multiple partial payments,

handling overpayments, and fixing confirmed payments require validation.

- **Source:** US009.
- **Priority:** High.
- **Acceptance criteria:** When recording a partial payment of 30.00 on an order of 100.00 with no previous payments, the system saves the payment and shows a balance of 70.00.

RF-09. View pending balances

The system must calculate the pending balance as the total of the order minus the sum of its confirmed payments.

The debt query must show the customer, the order code, the total, the paid amount, and the current balance. It must include delivered orders that still have debt.

- **Source:** US009.
- **Priority:** High.
- **Acceptance criteria:** A delivered order with a positive balance appears in the debt query; an order with a zero balance is excluded.

RF-10. Record income and expenses

The system must allow recording movements with ID, date, amount, type, and reason.

When an income corresponds to an order payment, it must keep the reference to the original payment and prevent that same payment from generating duplicate income entries.

- **Source:** US007 and project definition.
- **Priority:** High.
- **Acceptance criteria:** A payment linked to cash produces a single income entry for its amount. Viewing the payment again does not generate extra movements.

RF-11. View movements and balance

The system must allow selecting a date range and viewing its movements, total income, total expenses, and balance.

The range must include the start and end dates. The system must reject a start date that is later than the end date.

The balance will be equal to income minus expenses. This value represents the recorded movements and does not constitute a physical cash count.

- **Source:** US007 and project definition.
- **Priority:** High.
- **Acceptance criteria:** A period with income of 50.00 and expenses of 10.00 shows a balance of 40.00. A period with no movements shows zero totals.

RF-12. Generate reports

The system must allow viewing daily, weekly, and monthly reports of orders and financial activity. Each report must identify the type of query and the dates it covers.

The criteria for the start of the week and time selection of orders require validation. Financial amounts must correspond to the movements included in the period.

- **Source:** US005.
- **Priority:** Medium.
- **Acceptance criteria:** The report shows the applied date range and its results match the records that meet the defined criteria.

3.3. Performance Requirements

RNF-01. Operation response time

It is proposed that at least 95% of searches and saves finish in a maximum of two seconds per operation.

Measurement must be made from the user's request until the result is shown or the save is confirmed.

RNF-02. Report generation time

It is proposed that at least 95% of monthly reports are shown in a maximum of five seconds from their request.

Both goals require validation. Before using them as acceptance conditions, the computer, environment, volume of records, number of simultaneous users, and measurement procedure must be documented. These values do not represent results of executed tests.

3.4. Design Constraints

RD-01. Software organization

Development must apply Object-Oriented Programming and separate the responsibilities of presentation, business rules, and storage. Financial calculations must not be implemented only inside interface events.

RD-02. Technology platform

It is proposed to use Java and Swing for a local application. The Java version, dependencies, and compatible operating systems must be confirmed before delivery.

RD-03. Representation of amounts and dates

Amounts must be processed using decimal math to avoid differences caused by approximate binary representations.

It is proposed to use BigDecimal for money and LocalDate for dates. Currency, precision, and rounding rules require validation and must be applied uniformly.

RD-04. Independent verification

Operations for calculating totals, payments, balances, and cash balance must be testable without starting the graphical interface.

3.5. System Attributes

RNF-03. Security

The system must prevent administrative operations without a valid session. Passwords must be stored using a hash mechanism with an individual salt and must not appear in plain text in files, messages, or diagnostic logs.

The names employee, cashier, and manager describe business responsibilities. Differentiated permissions by role require additional specification.

RNF-04. Reliability and integrity

Recording a payment and its related income must be confirmed as one complete operation. If saving fails, the system must keep the previous state without leaving partially updated records.

In case of a read error, it must report the problem and avoid overwriting existing data with empty records.

RNF-05. Usability

Forms must identify mandatory fields and explain failed validations. The user must be able to fix an entry without re-entering the other valid data.

The system must ask for confirmation before discarding unsaved changes.

RNF-06. Maintainability

Business rules must allow independent tests of the interface and the storage mechanism. Replacing the storage system must not require changing the formulas for totals, balances, or cash balance.

RNF-07. Installation and portability

The delivery must include instructions with the environment version, dependencies, execution procedure, and data location.

Compatibility must be verified in the environments defined for delivery; compatibility with untested platforms will not be assumed.

3.6. Other Requirements

OR-01. Information preservation

It is proposed to keep customers, catalog, orders, payments, and movements after closing and restarting the application. Recovery must keep the IDs and relationships between records.

The storage mechanism and location require validation.

OR-02. Financial corrections

Cancellations, refunds, and modifications of confirmed payments must have rules that define their effect on orders and cash before being added to the scope.

OR-03. Document consistency

The specification, UML diagrams, and execution instructions must match the delivered behavior.

Changes must keep requirement IDs and update their affected references and tests.

4. Appendices

Formats of input and output

The formats must document the fields, data types, mandatory status, and validations of customers, catalog, orders, payments, and movements. They must also describe the information shown on the dashboard, debt queries, and reports.

Acceptance scenarios

- **CA-01. Calculation of an order:** Record two units at 15.00 and one unit at 20.00. The expected total is 50.00.
- **CA-02. Partial payment:** Record an order of 100.00 and a payment of 30.00. The expected balance is 70.00.
- **CA-03. Debt after delivery:** Mark the order from CA-02 as *Delivered*. The balance must remain at 70.00 and the order must continue appearing in the debt query.
- **CA-04. Balance of movements:** Query a period with income of 50.00 and expenses of 10.00. The expected balance is 40.00.
- **CA-05. Integrity on failures:** Simulate a failure when saving a payment linked to cash. Neither the payment nor its related income must be confirmed.
- **CA-06. Data recovery:** Once persistence is approved, close and open the application. Confirmed records must keep their IDs, relationships, and amounts.

These scenarios define expected results and do not constitute evidence of executed tests.